

Linux Command Reference Manual

Comprehensive Cheat Sheet & Terminal Configuration Guide | Derived from Hostinger Tutorials

This document serves as an exhaustive reference manual for system administrators, developers, and engineers working in Linux cloud ecosystems. It compiles essential utilities across categories ranging from directory navigation and core data extraction to network provisioning.

File & Directory Navigation

pwd Prints the full path of the current working directory.

Syntax: `pwd [options]`

Common Flags: -L (logical path with symlinks), -P (physical path resolving symlinks)

Practical Usage: `pwd -P`

ls Lists files and directories within a specified or current location.

Syntax: `ls [options] [directory]`

Common Flags: -l (long format detailing sizes, owners), -a (show hidden files), -h (human readable sizes)

Practical Usage: `ls -lah /var/www`

cd Changes the current working directory to a specified path.

Syntax: `cd [path]`

Common Flags: cd (home directory), cd .. (up one level), cd - (previous directory)

Practical Usage: `cd /var/www/html`

mkdir Creates one or more new empty directories.

Syntax: `mkdir [options] directory_name`

Common Flags: -p (create nested parent directories automatically without throwing errors), -m (set custom mode/permissions)

Practical Usage: `mkdir -p /path/to/new_folder`

rmdir Removes empty directories only.

Syntax: `rmdir [options] directory_name`

Common Flags: `-p` (remove explicit parent directories if they become empty)

Practical Usage: `rmdir -p backup/old`

rm Deletes files or directory structures permanently.

Syntax: `rm [options] file_or_dir`

Common Flags: `-r` (recursive for directories), `-f` (force extraction/removal without prompting), `-i` (prompt before delete)

Practical Usage: `rm -rf /tmp/cache_old`

File Management & Manipulation

cp Copies files or whole folders from a source to a destination.

Syntax: `cp [options] source target`

Common Flags: `-R` (recursive for folder trees), `-i` (interactive prompt before overwrite), `-v` (verbose log output)

Practical Usage: `cp -R /src/folder /dest/folder`

mv Moves or renames files and folders across locations.

Syntax: `mv [options] source target`

Common Flags: `-v` (explain what is being done), `-n` (do not overwrite existing files)

Practical Usage: `mv old_name.txt new_name.txt`

touch Creates a new empty file or updates access/modified timestamps of existing files.

Syntax: `touch [options] filename`

Common Flags: `-c` (do not create if file doesn't exist), `-a` (change access time only), `-m` (modification time only)

Practical Usage: `touch config.json`

file Inspects and identifies the structural file type based on contents/magic numbers.

Syntax: `file [options] filename`

Common Flags: `-k` (keep going, look past first match), `-i` (output MIME type strings)

Practical Usage: `file image.png`

zip / unzip Compresses files into standard ZIP archives or extracts compressed files out.

Syntax: `zip [opts] archive.zip files | unzip archive.zip`

Common Flags: -r (zip recursively), -e (encrypt zip archive with password input)

Practical Usage: `zip -re archive.zip document_folder/`

tar Archives files into tarballs, often with gzip or bzip2 compression utilities.

Syntax: `tar [options] archive.tar.gz targets`

Common Flags: -c (create), -x (extract), -f (specify file), -z (gzip tool filter), -v (verbose)

Practical Usage: `tar -czvf backup.tar.gz /home/user`

Text Editors & Viewing Content

cat Concatenates and displays file contents straight to the standard output terminal.

Syntax: `cat [options] filename`

Common Flags: -n (number all output lines), -T (display tab characters as ^I)

Practical Usage: `cat -n /etc/hosts`

nano A simple, user-friendly terminal-based visual text editor.

Syntax: `nano filename`

Common Flags: Built-in interactive shortcut menus: Ctrl+O (Save), Ctrl+X (Exit)

Practical Usage: `nano /etc/nginx/nginx.conf`

vi / vim Advanced modal terminal text editor preferred for extensive code/configuration changes.

Syntax: `vi filename`

Common Flags: Modes: Command mode, Insert mode (i), Colon mode (:w to write, :q to quit)

Practical Usage: `vi /etc/fstab`

head Outputs the very first beginning lines of a specified file.

Syntax: `head [options] filename`

Common Flags: -n [number] (print specific count of lines, defaults to first 10 items)

Practical Usage: `head -n 20 error.log`

tail Outputs the absolute final tail-end lines of a targeted text file.

Syntax: `tail [options] filename`

Common Flags: `-n [number]` (print custom count of lines), `-f` (live append track follow loop logs dynamically)

Practical Usage: `tail -f /var/log/syslog`

less Interactive reader that allows back/forward page scrolls inside oversized files.

Syntax: `less filename`

Common Flags: Navigation keys: Space (Page Down), B (Page Up), `/pattern` (Search forward)

Practical Usage: `less /var/log/auth.log`

Text Processing & Filtering

grep Searches for matching text regular expression pattern strings across targeted files.

Syntax: `grep [options] pattern file`

Common Flags: `-i` (ignore capitalization case), `-r` (recursive directories), `-v` (invert match search logic)

Practical Usage: `grep -ri 'failed' /var/log/`

sed Stream editor tool for parsing, finding, transforming, and filtering stream patterns.

Syntax: `sed [options] 'script' file`

Common Flags: `s/old/new/g` (substitute glob pattern matches inline), `-i` (modify text files in-place)

Practical Usage: `sed -i 's/localhost/127.0.0.1/g' config.env`

awk Powerful text scanning and column processing language tailored to data analytics reporting.

Syntax: `awk 'program' file`

Common Flags: `-F` (specify column delimiter split characters), defaults to spacing blocks

Practical Usage: `awk -F: '{print $1, $3}' /etc/passwd`

sort Sorts input text file lines alphabetically, numerically, or custom rules.

Syntax: `sort [options] file`

Common Flags: `-n` (numeric sorting sequence evaluation), `-r` (reverse order sequencing output results)

Practical Usage: `sort -n metrics.txt`

cut Extracts section clips out from every individual text line in a file structured grid.

Syntax: `cut [options] file`

Common Flags: -d (custom fields separator character delimiter), -f (fields position arrays selector)

Practical Usage: `cut -d', ' -f1 data.csv`

diff Compares two targeted text file buffers line by line to calculate visual changes.

Syntax: `diff [options] file1 file2`

Common Flags: -w (ignore all whitespace changes), -u (unified layout output context viewer format)

Practical Usage: `diff -u config.orig config.new`

System Administration & Account Controls

sudo Executes a standalone console application with administrative or security root privileges.

Syntax: `sudo command`

Common Flags: -i (simulate login shell environment for target user root account environment)

Practical Usage: `sudo systemctl restart nginx`

su Switches the active shell identity over to another user environment session.

Syntax: `su [options] [username]`

Common Flags: - (provide standard target user environment variables and configuration startup state)

Practical Usage: `su - admin`

chmod Modifies system object access permission bit indicators.

Syntax: `chmod [options] mode file`

Common Flags: Octal values: 7 (rwx), 6 (rw-), 4 (r--). Parameters: -R (recursively apply flags downstream)

Practical Usage: `chmod 755 script.sh`

chown Changes explicit owner and grouping metrics assignment of filesystem targets.

Syntax: `chown [options] owner[:group] target`

Common Flags: -R (recursively apply file ownership parameters down directories nested path structures)

Practical Usage: `chown -R www-data:www-data /var/www`

whoami Prints out the current effective user login registry identity string token.

Syntax: `whoami`

Common Flags: None

Practical Usage: `whoami`

hostname Displays or provisions updates to the current system hostname network label tag.

Syntax: `hostname [options]`

Common Flags: `-I` (displays all local network IP interfaces addresses assigned cleanly)

Practical Usage: `hostname -I`

System Performance & Process Management

top / htop

Dynamic interactive process resource usage monitor tracks active processes and hardware stats.

Syntax: `top | htop`

Common Flags: `htop` features cleaner color bars and easy cursor navigation lists (if installed)

Practical Usage: `htop`

ps Provides a static snapshot report tracking currently running user/system processes.

Syntax: `ps [options]`

Common Flags: `aux` (all user items listed, process owner labels, un-entangled terminal wrappers)

Practical Usage: `ps aux | grep node`

kill Sends a targeted operational signal (like terminate) to a running process ID (PID).

Syntax: `kill [options] PID`

Common Flags: `-9` (SIGKILL immediately force closure), `-15` (SIGTERM clean exit sequence default)

Practical Usage: `kill -9 1234`

systemctl Controls, inspects, starts, halts, and provisions state for systemd service daemons.

Syntax: `systemctl [action] service`

Common Flags: Actions: `start`, `stop`, `restart`, `status`, `enable` (boot run auto), `disable`

Practical Usage: `systemctl status docker`

df Reports storage space usage statistics for all active mounted filesystems.

Syntax: `df [options]`

Common Flags: `-h` (human readable output sizing formatting scales using MB/GB units cleanly)

Practical Usage: `df -h`

du Estimates and tallies disk capacity storage space consumed by targeted directories.

Syntax: `du [options] [path]`

Common Flags: `-h` (human readable), `-s` (summarize root total volume without individual items list metrics)

Practical Usage: `du -sh /var/log`

Networking & External Downloads

ping Sends ICMP echo packets to evaluate network path connectivity and host latency metrics.

Syntax: `ping [options] destination`

Common Flags: `-c [count]` (limits maximum amount of packet ping sequences sent before stop)

Practical Usage: `ping -c 4 google.com`

wget Non-interactive network downloader utility that extracts files directly from web hosts.

Syntax: `wget [options] URL`

Common Flags: `-O [filename]` (custom output save mapping path), `-c` (resume half-completed stream files)

Practical Usage: `wget -O package.tar.gz https://example.com/file`

curl Transfers data to or from a network server using supported application protocols (HTTP/FTP).

Syntax: `curl [options] URL`

Common Flags: `-I` (fetch header metadata only), `-o [file]` (save body content target), `-L` (follow redirect paths)

Practical Usage: `curl -IL https://hostinger.com`

ssh Establishes an encrypted secure shell tunnel login connection to remote host nodes.

Syntax: `ssh [options] user@host`

Common Flags: `-p [port]` (connect target alternate port structure overrides), `-i [identity_file]` (private key map)

Practical Usage: `ssh -i ~/.ssh/id_rsa root@192.168.1.50`

scp

Securely copies files between networked endpoints over an encrypted SSH protocol link layer.

Syntax: `scp [options] source target`

Common Flags: `-P [port]` (override SSH port parameters), `-r` (recursive directory tree transfers)

Practical Usage: `scp -P 2222 local.txt user@remote:/path/`

rsync

Advanced incremental file syncing engine optimized for rapid delta local/remote copy tracking.

Syntax: `rsync [options] source target`

Common Flags: `-a` (archive mode preserves attributes), `-v` (verbose output logs), `-z` (compress packet streams)

Practical Usage: `rsync -avz /src/ backup/user@host:/dest/`